

File: src\data\modular_arithmetic.py

```
from torch import long, tensor
from torch.utils.data import Dataset, DataLoader, random_split

def generate_pairs(number):
    pairs = []
    for i in range(number):
        for j in range(number):
            pairs.append((i, j, (i + j) % number))
    return pairs

def get_dataloaders(number, batch_size):
    modular_arithmetic_dataset = ModularArithmeticDataset(number)
    train_size = int(0.3 * len(modular_arithmetic_dataset))
    test_size = len(modular_arithmetic_dataset) - train_size

    train_dataset, test_dataset = random_split(modular_arithmetic_dataset, [train_size,
    test_size])
    train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_dataloader = DataLoader(test_dataset, batch_size=batch_size)

    return train_dataloader, test_dataloader

class ModularArithmeticDataset(Dataset):
    def __init__(self, number):
        self.pairs = generate_pairs(number)

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        return (self.get_tensor(self.pairs[idx]), self.pairs[idx][2])

    def get_tensor(self, item):
        sequence = [item[0], item[1], 97]
        return tensor(sequence)

if __name__ == "__main__":
    print(tensor([5, 3, 8, 97]))
```

File: src\models\transformer.py

```
from torch import arange, nn
import torch

class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(3, d_model)

        self.query = nn.Linear(d_model, d_model, bias=False)
        self.key = nn.Linear(d_model, d_model, bias=False)
        self.value = nn.Linear(d_model, d_model, bias=False)
```

```

self.mlp_in = nn.Linear(d_model, 4 * d_model, bias=False)
self.mlp_activation = nn.ReLU()
self.mlp_out = nn.Linear(4 * d_model, d_model, bias=False)

self.output_head = nn.Linear(d_model, vocab_size, bias=False)

self.dropout1 = nn.Dropout(0.0)
self.dropout2 = nn.Dropout(0.0)

def forward(self, x):
    token_vectors = self.token_embedding(x)
    position_vectors = self.position_embedding(arange(x.size(1), device=x.device))
    combined_vector = token_vectors + position_vectors
    query_vector = self.query(combined_vector)
    key_vector = self.key(combined_vector)
    value_vector = self.value(combined_vector)

    scores = torch.matmul(query_vector, key_vector.transpose(-2, -1)) /
    (query_vector.size(-1) ** 0.5)
    attention_weights = torch.softmax(scores, dim=-1)
    attention_values = combined_vector + self.dropout1(torch.matmul(attention_weights,
    value_vector))
    mlp_output = attention_values +
    self.dropout2(self.mlp_out(self.mlp_activation(self.mlp_in(attention_values))))

    logits = self.output_head(mlp_output)

    return logits

if __name__ == "__main__":
    model = Transformer(vocab_size=98, d_model=128)

    x = torch.eye(2, 3, dtype=torch.long)
    model.forward(x)

```

File: src\plot_results.py

```

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os

from predictors.l2_norm import compute_noise_floor, detect_ma_of_ma_zero_crossing

# Look for files in project root (parent of src/)
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

# Load data already computed and saved by train.py – no re-run needed.
epoch_grid = np.load("epoch_grid.npy")
slow_ma = np.load("slow_ma.npy")

```

```

fast_ma_of_slow_ma = np.load("fast_ma_of_slow_ma.npy")
diff = np.load("ma_of_ma_diff.npy")
train_acc = np.load("train_acc_history.npy")
test_acc = np.load("test_acc_history.npy")
num_epochs = len(train_acc)

SKIP_EPOCHS = 100
QUIET_EPOCH_CUTOFF = 90

# Noise floor printed for context only – the trigger itself is a plain zero
# crossing now, not a magnitude threshold.
noise_floor = compute_noise_floor(diff, epoch_grid,
quiet_epoch_cutoff=QUIET_EPOCH_CUTOFF)
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, diff,
skip_epochs=SKIP_EPOCHS)

# Plot 1: Slow MA vs. Fast MA of Slow MA, with the trigger point marked
fig, ax = plt.subplots(figsize=(12, 7))

ax.plot(epoch_grid, slow_ma, color="purple", linewidth=3.5, label="Slow MA (w=200)")
ax.plot(epoch_grid, fast_ma_of_slow_ma, color="orange", linewidth=3.5, label="Fast
MA of Slow MA (w=20)")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax.scatter(trigger_epoch, slow_ma[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax.set_xscale("log")
ax.set_xlabel("Epoch (log scale)", fontsize=12)
ax.set_ylabel("L2 Norm", fontsize=12)
ax.set_title("Slow MA vs. Fast MA of Slow MA – Zero-Crossing Trigger", fontsize=13)
ax.legend(loc="upper right", fontsize=11)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_slow_ma_crossover.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_crossover.png")

# Plot 2: The difference curve itself, with the zero-crossing trigger marked
fig2, ax2 = plt.subplots(figsize=(12, 7))

ax2.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax2.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax2.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax2.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax2.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax2.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax2.set_xscale("log")

```

```

ax2.set_xlabel("Epoch (log scale)", fontsize=12)
ax2.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax2.set_title("Difference: Fast MA of Slow MA minus Slow MA - Zero-Crossing Trigger", fontsize=13)
ax2.legend(loc="upper right", fontsize=11)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_slow_ma_diff.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff.png")

# Plot 3: Same difference curve as Plot 2, but on a LINEAR x-axis instead of log.
fig3, ax3 = plt.subplots(figsize=(12, 7))

ax3.plot(epoch_grid, diff, color="darkred", linewidth=3.5)
ax3.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax3.fill_between(epoch_grid, 0, diff, where=(diff > 0), alpha=0.2, color="green")
ax3.fill_between(epoch_grid, 0, diff, where=(diff <= 0), alpha=0.2, color="red")

if trigger_epoch is not None:
    trigger_idx = np.argmin(np.abs(epoch_grid - trigger_epoch))
    ax3.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax3.scatter(trigger_epoch, diff[trigger_idx], color="red", zorder=5, s=150,
marker="*",
                label=f"Trigger (epoch {trigger_epoch:.0f})")

ax3.set_xlabel("Epoch (linear scale)", fontsize=12)
ax3.set_ylabel("Fast MA of Slow MA - Slow MA", fontsize=12)
ax3.set_title("Difference: Fast MA of Slow MA minus Slow MA - Linear Scale",
fontsize=13)
ax3.legend(loc="upper right", fontsize=11)
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_slow_ma_diff_linear.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_slow_ma_diff_linear.png")

# Plot 4: Difference curve (linear x-axis) overlaid on the grokking curve
# (train/test accuracy). Different scales (accuracy 0-1 vs. diff ~0.00-0.02),
# so accuracy uses the left y-axis and the diff uses a twin right y-axis.
grok_epoch = int(np.argmax(test_acc > 0.9))

fig4, ax4 = plt.subplots(figsize=(12, 7))

ax4.plot(range(1, num_epochs + 1), train_acc, color="steelblue", linewidth=2.5,
label="Train Accuracy")
ax4.plot(range(1, num_epochs + 1), test_acc, color="seagreen", linewidth=2.5,
label="Test Accuracy")
ax4.set_xlabel("Epoch (linear scale)", fontsize=12)
ax4.set_ylabel("Accuracy", fontsize=12)
ax4.set_ylim(-0.05, 1.05)

ax4b = ax4.twinx()
ax4b.plot(epoch_grid, diff, color="darkred", linewidth=3, label="Fast MA of Slow MA
- Slow MA", alpha=0.85)
ax4b.axhline(y=0, color="black", linestyle="--", linewidth=1)
ax4b.set_ylabel("MA-of-MA Difference", fontsize=12)

```

```

ax4.axvline(x=grok_epoch, color="seagreen", linestyle=":", linewidth=2, alpha=0.7)
if trigger_epoch is not None:
    ax4.axvline(x=trigger_epoch, color="red", linestyle="--", linewidth=2)
    ax4b.scatter(trigger_epoch, diff[np.argmin(np.abs(epoch_grid - trigger_epoch))],
color="red", zorder=5, s=150, marker="*", label=f"Trigger (epoch
{trigger_epoch:.0f})")

ax4.set_title(f"MA-of-MA Difference vs. Grokking Curve (Linear Scale) – Grok epoch
{grok_epoch}", fontsize=13)
lines4, labels4 = ax4.get_legend_handles_labels()
lines4b, labels4b = ax4b.get_legend_handles_labels()
ax4.legend(lines4 + lines4b, labels4 + labels4b, loc="center right", fontsize=10)
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("ma_of_ma_diff_vs_grokking_linear.png", dpi=150, bbox_inches="tight")
print("[OK] Plot saved to ma_of_ma_diff_vs_grokking_linear.png")

print(f"\nNoise floor (context only): {noise_floor:.6f}")
if trigger_epoch is not None:
    print(f"Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
else:
    print("No zero-crossing detected")

```

File: src\predictors\dropout.py

```
import torch
```

```

def compute_accuracy(model, data_loader):
    device = next(model.parameters()).device
    total_correct = 0
    total_samples = 0
    for x, y in data_loader:
        x, y = x.to(device), y.to(device)
        with torch.no_grad():
            logit = model.forward(x)
            equal_sign_logit = logit[:, 2, :]
            predicted = equal_sign_logit.argmax(dim=1)
            total_correct += (predicted == y).sum().item()
            total_samples += len(y)
    return total_correct / total_samples if total_samples > 0 else 0.0

def compute_dropout_gap(model, data_loader, dropout_rate):
    model.train()
    model.dropout1.p = dropout_rate
    model.dropout2.p = dropout_rate
    train_accuracy = compute_accuracy(model, data_loader)

    model.eval()
    model.dropout1.p = 0.0
    model.dropout2.p = 0.0
    eval_accuracy = compute_accuracy(model, data_loader)

    return train_accuracy - eval_accuracy, train_accuracy, eval_accuracy

```

File: src\predictors\l2_norm.py

```
import numpy as np
import torch
from scipy.ndimage import uniform_filter1d

def compute_l2_norm(model):
    all_params = torch.cat([p.flatten() for p in model.parameters()])
    l2_norm = torch.norm(all_params).item()
    return l2_norm

def compute_l2_norm_rate_of_decline(l2_norm_history):
    return np.diff(l2_norm_history) *-1

def detect_l2_norm_drop(rate_of_decline, threshold=0.05, skip_epochs=200):
    for i, rate in enumerate(rate_of_decline):
        if i < skip_epochs: # Skip early epochs
            continue
        if rate > threshold:
            return i
    return None

def compute_acceleration(rate_of_decline):
    """
    Compute the second derivative of L2 norm (acceleration).
    acceleration[i] = rate[i] - rate[i-1]
    Positive = decay rate is increasing (speeding up)
    Negative = decay rate is decreasing (slowing down)
    Sign change = inflection point (change in decay character)
    """
    acceleration = np.diff(rate_of_decline)
    return acceleration

def detect_inflection(acceleration, skip_epochs=200):
    """
    Detect inflection point where acceleration changes sign.
    This is where the decay pattern changes character – exactly what happens
    before/during the grok transition.
    Returns the epoch index where the sign flip occurs, or None.
    """
    if len(acceleration) <= skip_epochs:
        return None

    # Check from skip_epochs onwards for sign changes
    for i in range(skip_epochs, len(acceleration)):
        # acceleration[i-1] * acceleration[i] < 0 means opposite signs (sign flip)
        if i > 0 and acceleration[i-1] * acceleration[i] < 0:
            return i

    return None

def apply_moving_average(data, window_size=50):
    """
```

Apply moving average smoothing to reduce noise.
 window_size: number of epochs to average over (default 50).
 Returns smoothed array (same length as input, padded at edges).
 """
 return uniform_filter1d(np.array(data), size=window_size, mode='nearest')

```
def resample_to_log_uniform_grid(data, num_points=None):
    """
    Resample data onto a grid uniformly spaced in log10(epoch) – i.e. the
    SAME spacing the data has when viewed on a log-x plot.
    Without this, a fixed window_size (in raw epoch index) covers a huge
    visual chunk of the plot near epoch 1 and a tiny sliver near epoch N,
    which is why smoothing looked over-smoothed early and scribbly late.
    Returns (epoch_grid, resampled_data) where epoch_grid is uniform in
    log10-space (non-uniform in linear epoch terms).
    """
    data = np.array(data)
    num_epochs = len(data)
    epochs = np.arange(1, num_epochs + 1)
    log_epochs = np.log10(epochs)

    if num_points is None:
        num_points = num_epochs

    log_grid = np.linspace(log_epochs[0], log_epochs[-1], num_points)
    resampled = np.interp(log_grid, log_epochs, data)
    epoch_grid = 10 ** log_grid

    return epoch_grid, resampled

def compute_fast_slow_moving_averages(l2_norm_history, fast_window=50,
slow_window=200, num_points=None):
    """
    Compute two moving averages on a LOG-EPOCH-UNIFORM grid, so the window
    covers equal VISUAL width everywhere on a log-x plot (fixes both the
    over-smoothing near epoch 1 and the scribbling near epoch N).

    Steps:
    1. Log-transform L2 norm values (handles the exponential decay in y).
    2. Resample onto a grid uniform in log10(epoch) (handles the log-x axis).
    3. Apply the moving average on that resampled grid.
    4. Convert back to linear L2 norm space.

    fast_window / slow_window are now measured in grid points, which (since
    the grid is log-uniform) corresponds to a constant *proportional* width
    in epochs everywhere along the curve.

    Returns (epoch_grid, fast_ma, slow_ma) – always plot fast_ma/slow_ma
    against epoch_grid, not against range(1, num_epochs+1).
    """
    l2_norm_history = np.array(l2_norm_history)
    log_l2_norm = np.log(l2_norm_history)

    epoch_grid, log_l2_norm_resampled = resample_to_log_uniform_grid(log_l2_norm,
num_points=num_points)
```

```

fast_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=fast_window)
slow_ma_log = apply_moving_average(log_l2_norm_resampled, window_size=slow_window)

fast_ma = np.exp(fast_ma_log)
slow_ma = np.exp(slow_ma_log)

return epoch_grid, fast_ma, slow_ma

def detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100):
    """
    Detect where fast MA crosses above slow MA, searching along the
    log-uniform grid. skip_epochs is a real epoch value (not a grid index) –
    crossovers before this epoch are ignored (initialization noise).
    Returns the real epoch (float) of the crossover, or None.
    """
    if len(fast_ma) != len(slow_ma):
        return None

    for i in range(len(fast_ma) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        # Previous: fast <= slow, Current: fast > slow (crossover from below)
        if fast_ma[i] <= slow_ma[i] and fast_ma[i + 1] > slow_ma[i + 1]:
            return epoch_grid[i + 1]

    return None

def compute_ma_of_slow_ma(slow_ma, fast_window=20):
    """
    Apply a second, faster-window moving average on top of the already-smoothed
    slow_ma. Since slow_ma has already filtered out raw L2 norm noise, this
    reveals slow_ma's OWN turning points cleanly – the difference between the
    two tracks how sharply slow_ma is currently bending.
    Returns (fast_ma_of_slow_ma, diff) where diff = fast_ma_of_slow_ma - slow_ma.
    """
    fast_ma_of_slow_ma = apply_moving_average(slow_ma, window_size=fast_window)
    diff = fast_ma_of_slow_ma - slow_ma
    return fast_ma_of_slow_ma, diff

def compute_noise_floor(diff, epoch_grid, quiet_epoch_cutoff=90):
    """
    Estimate the normal noise level of the MA-of-MA difference from the quiet
    early-training region (before any real dynamics begin), so later spikes
    can be judged against what "flat" actually looks like for this run.
    """
    quiet_mask = epoch_grid < quiet_epoch_cutoff
    return np.std(diff[quiet_mask])

def detect_ma_of_ma_trigger(epoch_grid, diff, noise_floor, threshold_multiplier=10,
skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff climbs past

```


threshold_multiplier x noise_floor – the rising edge of a real signal. Causal: only needs the noise floor (known early) and points seen so far, unlike picking the curve's peak, which requires seeing the whole future. Returns the real epoch (float) of the trigger, or None.

```
"""
threshold = threshold_multiplier * noise_floor
for i in range(len(diff)):
    if epoch_grid[i] < skip_epochs:
        continue
    if diff[i] > threshold:
        return epoch_grid[i]
return None
```

```
def detect_ma_of_ma_zero_crossing(epoch_grid, diff, skip_epochs=100):
    """
    Fire on the first epoch (after skip_epochs) where diff crosses from
    positive to negative (the first "green -> red" zero crossing).
    Confirmed on two separate training runs to land in a consistent,
    unambiguous spot even though the crossing's magnitude is weak.
    Returns the real epoch (float) of the crossing, or None.
    """
    for i in range(len(diff) - 1):
        if epoch_grid[i] < skip_epochs:
            continue
        if diff[i] >= 0 and diff[i + 1] < 0:
            return epoch_grid[i + 1]
    return None
```

File: src\train.py

```
import torch
from torch import nn
from torch.optim import AdamW
import numpy as np
import os
from predictors.l2_norm import (
    compute_l2_norm,
    compute_fast_slow_moving_averages,
    detect_ma_crossover,
    compute_ma_of_slow_ma,
    compute_noise_floor,
    detect_ma_of_ma_zero_crossing,
)
from data.modular_arithmetic import get_data loaders
from models.transformer import Transformer
from predictors.dropout import compute_dropout_gap

# Save results to project root
project_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
os.chdir(project_root)

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print("Device:", device)
```

```

data_loader = get_data loaders(97, batch_size=int(0.3 * 97 * 97))
model = Transformer(vocab_size=98, d_model=128).to(device)
optimizer = AdamW(model.parameters(), lr=1e-3, weight_decay=1.0)

print("Optimizer:", optimizer)
cross_entropy_loss = nn.CrossEntropyLoss()

num_epochs = 10000
train_acc_history = []
test_acc_history = []
loss_history = []
l2_norm_history = []
dropout_gap_epochs = []
dropout_gap_history = []
dropout_train_acc_history = []
dropout_eval_acc_history = []

for epoch in range(num_epochs):
    total_correct = 0
    total_samples = 0
    for x, y in data_loader[0]:
        x, y = x.to(device), y.to(device)
        logit = model.forward(x)
        equal_sign_logit = logit[:, 2, :]

        loss = cross_entropy_loss(equal_sign_logit, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        predicted = equal_sign_logit.argmax(dim=1)
        total_correct += (predicted == y).sum().item()
        total_samples += len(y)

    test_total_correct = 0
    test_total_samples = 0

    for x_test, y_test in data_loader[1]:
        x_test, y_test = x_test.to(device), y_test.to(device)
        logit_test = model.forward(x_test)
        equal_sign_logit_test = logit_test[:, 2, :]
        predicted_test = equal_sign_logit_test.argmax(dim=1)
        test_total_correct += (predicted_test == y_test).sum().item()
        test_total_samples += len(y_test)

    train_acc_history.append(total_correct / total_samples)
    test_acc_history.append(test_total_correct / test_total_samples)
    loss_history.append(loss.item())
    l2_norm_history.append(compute_l2_norm(model))
    if (epoch + 1) % 100 == 0:
        compute_l2 = l2_norm_history[-1]

        # Dropout Gap Predictor: tracked only every 100th epoch, because it needs
        # two full passes over the test set (dropout ON, dropout OFF) – doing this
        # every single epoch would slow training down a lot for no real benefit.

```

```

dropout_gap, dropout_train_acc, dropout_eval_acc = compute_dropout_gap(
    model, data_loader[1], dropout_rate=0.9
)
dropout_gap_epochs.append(epoch + 1)
dropout_gap_history.append(dropout_gap)
dropout_train_acc_history.append(dropout_train_acc)
dropout_eval_acc_history.append(dropout_eval_acc)

# compute_dropout_gap() leaves the model in eval() mode with p reset to
# 0.0. Training must resume in train() mode, so we restore it here before
# the next epoch's training pass begins.
model.train()

print(f"Epoch [{epoch + 1}/{num_epochs}], Loss: {loss.item():.4f}, Train Accuracy:
{train_acc_history[-1]:.4f}, Test Accuracy: {test_acc_history[-1]:.4f}, L2 Norm:
{compute_l2:.4f}, Dropout Gap: {dropout_gap:.4f}")

np.save("train_acc_history.npy", train_acc_history)
np.save("test_acc_history.npy", test_acc_history)
np.save("loss_history.npy", loss_history)
np.save("l2_norm_history.npy", l2_norm_history)
np.save("dropout_gap_epochs.npy", dropout_gap_epochs)
np.save("dropout_gap_history.npy", dropout_gap_history)
np.save("dropout_train_acc_history.npy", dropout_train_acc_history)
np.save("dropout_eval_acc_history.npy", dropout_eval_acc_history)
print("Training data saved:")
print(" - train_acc_history.npy")
print(" - test_acc_history.npy")
print(" - loss_history.npy")
print(" - l2_norm_history.npy")
print(" - dropout_gap_epochs.npy")
print(" - dropout_gap_history.npy")
print(" - dropout_train_acc_history.npy")
print(" - dropout_eval_acc_history.npy")

# L2 Norm Predictor: Moving Average Crossover Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: Moving Average Crossover")
print("="*60)

# Compute fast and slow moving averages on a log-epoch-uniform grid
# (matches the log-x plot's visual spacing, so the window covers equal
# visual width everywhere instead of scribbling at high epochs)
# Fast MA (window=50 grid points): responsive to recent changes
# Slow MA (window=200 grid points): captures overall trend
epoch_grid, fast_ma, slow_ma = compute_fast_slow_moving_averages(l2_norm_history,
fast_window=50, slow_window=200)

# Detect crossover point: where fast MA crosses above slow MA
detection_epoch = detect_ma_crossover(epoch_grid, fast_ma, slow_ma, skip_epochs=100)

# Find when test accuracy actually jumps (crosses 90%)
grok_epoch = np.argmax(np.array(test_acc_history) > 0.9)

# Save data for plotting

```

```

np.save("epoch_grid.npy", epoch_grid)
np.save("fast_ma.npy", fast_ma)
np.save("slow_ma.npy", slow_ma)

# Report results
print(f"\nDetection Results:")
if detection_epoch is not None:
    lead_time = grok_epoch - detection_epoch
    print(f"  MA Crossover epoch: {detection_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {lead_time:.1f} epochs")
else:
    print("  No MA crossover detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

# Debug stats
print(f"\nL2 Norm Stats:")
print(f"  Min: {np.min(l2_norm_history):.4f}")
print(f"  Max: {np.max(l2_norm_history):.4f}")
print(f"  Final: {l2_norm_history[-1]:.4f}")

# L2 Norm Predictor: MA-of-MA Zero-Crossing Trigger Strategy
print("\n" + "="*60)
print("L2 NORM PREDICTOR: MA of Slow MA – Zero-Crossing Trigger")
print("="*60)

# Apply a second, faster-window MA on top of slow_ma itself. slow_ma has
# already filtered out raw L2 norm noise, so this reveals slow_ma's own
# turning points cleanly instead of chasing noise in the raw signal.
fast_ma_of_slow_ma, ma_of_ma_diff = compute_ma_of_slow_ma(slow_ma, fast_window=20)

# Noise floor: how big this difference is during the quiet early-training
# region, before any real dynamics begin – printed for context only, no
# longer used to decide the trigger (see detect_ma_of_ma_zero_crossing).
noise_floor = compute_noise_floor(ma_of_ma_diff, epoch_grid, quiet_epoch_cutoff=90)

# Trigger: first epoch (after skip_epochs) where the difference crosses from
# positive to negative. Confirmed on two runs to land in a consistent,
# unambiguous spot even though the crossing itself is weak in magnitude.
trigger_epoch = detect_ma_of_ma_zero_crossing(epoch_grid, ma_of_ma_diff,
skip_epochs=100)

np.save("fast_ma_of_slow_ma.npy", fast_ma_of_slow_ma)
np.save("ma_of_ma_diff.npy", ma_of_ma_diff)

print(f"\nNoise floor: {noise_floor:.6f}")
if trigger_epoch is not None:
    trigger_lead_time = grok_epoch - trigger_epoch
    print(f"  Trigger epoch (first zero-crossing): {trigger_epoch:.1f}")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")
    print(f"  Lead time: {trigger_lead_time:.1f} epochs")
else:
    print("  No trigger detected")
    print(f"  Grok epoch (test acc > 90%): {grok_epoch}")

```

```
# Dropout Predictor: gap tracked every 100th epoch during training (see loop
# above). We simply report the last tracked value here – no need to run
# compute_dropout_gap() a second time on the already-trained model.
print("\n" + "="*60)
print("DROPOUT PREDICTOR: Gap tracked every 100 epochs")
print("="*60)
print(f"\nFinal Dropout Gap Check (rate=0.9, epoch {dropout_gap_epochs[-1]}):")
print(f"  Accuracy with dropout (train mode, p=0.9):
{dropout_train_acc_history[-1]:.4f}")
print(f"  Accuracy without dropout (eval mode, p=0.0):
{dropout_eval_acc_history[-1]:.4f}")
print(f"  Gap: {dropout_gap_history[-1]:.4f}")
```